Your points are stored in the table, so when you click the point → you are actually clicking a
Feature whose attributes are your saved data.

Step-1: State for selected feature

```
var selectedFeature by remember { mutableStateOf<Feature?>(null) }
```

Step-2: Detect tap and identify feature

```
val scope = rememberCoroutineScope()

MapView(
    modifier = Modifier.fillMaxSize(),
    onSingleTapConfirmed = { tapEvent ->
        scope.launch {
            val result = mapViewProxy.identifyLayer(
                layer = featureCollectionLayer,   // your layer
                screenPoint = tapEvent.screenCoordinate,
                tolerance = 15.0,
                returnPopupsOnly = false,
                maximumResults = 1
            )

            if (result.geoElements.isNotEmpty()) {
                selectedFeature = result.geoElements[0] as Feature
            } else {
                selectedFeature = null
            }
        }
    }
)
```

Now whenever you tap a point, selectedFeature will contain that point’s data.

Step-3: Show stored data on screen (Dialog example)

```
if (selectedFeature != null) {
    AlertDialog(
        onDismissRequest = { selectedFeature = null },
        title = { Text("Point Details") },
        text = {
            Column {
                selectedFeature!!.attributes.forEach { (key, value) ->
                    Text("$key : $value")
                }
            }
        },
```

```
        confirmButton = {
            TextButton(onClick = { selectedFeature = null }) {
                Text("Close")
            }
        }
    )
}
```

Now your full flow becomes:

You draw point with GeometryEditor
↓
You save attributes in FeatureCollectionTable
↓
You tap the point
↓
identifyLayer() finds the Feature
↓
Feature.attributes is read
↓
Dialog / Screen shows stored data

If you want specific fields instead of looping:

```
val name = selectedFeature!!.attributes["name"]
val type = selectedFeature!!.attributes["type"]

Text("Name : $name")
Text("Type : $type")
```

This is exactly how professional GIS apps show “click a point → show its data”.
Your table structure is already correct; you just needed the identifyLayer() + UI binding.

tolerance = 15.0,

what is its use?

tolerance = 15.0 means how close to the point the user has to tap for the app to consider it a valid click on that feature.

It is measured in screen pixels, not in meters or map units.

So:

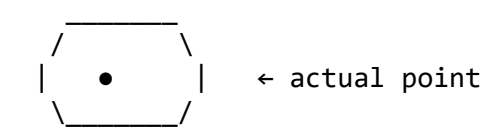
tolerance = 5.0 → user must tap very precisely on the point

tolerance = 15.0 → normal, comfortable touch accuracy

tolerance = 30.0 → very forgiving, even a rough tap will select the point

On mobile screens, users rarely tap exactly on a tiny point symbol, so tolerance is critical.

Think of it like this:
You are creating an invisible circle around every point:



If the tap falls anywhere inside that circle → the feature is selected.

Without tolerance, users would have to tap the exact pixel of the point, which is almost impossible on touch devices.

In ArcGIS terms:

```
identifyLayer(
    layer = featureCollectionLayer,
    screenPoint = tapEvent.screenCoordinate,
    tolerance = 15.0,        // search radius in pixels
    returnPopupsOnly = false,
    maximumResults = 1
)
```

So 15.0 means:

“Search within 15 pixels around the tap for any feature.”

For your use case (clicking drawn points):

12–20 px is ideal

15 px is a professional default

This is why your point becomes easy to select even if the tap is slightly off.